

Proposal: Adding the Variable-Length UBX Messages

Prepared for: SparkFun u-blox GNSS v4 migration **Scope:** AGENTS.md section “Adding the variable-length UBX messages” (NAV-SAT worked example) **Status:** Proposal only — no code changes have been made. This document is for review.

1. Summary

NAV-SAT (and the similarly-shaped NAV-SIG) are still on the old v3-style packetUBXNAVSA pointer / hand-written processUBXpacket() switch case, because they don’t fit the current ubxMessage model: a fixed-size payload described by a single flat ubxField[] table. NAV-SAT’s payload is an 8-byte header (iTOW, version, numSvs) followed by 0-255 repeated 12-byte per-satellite blocks.

The core finding of this analysis is that the existing registry machinery already tolerates variable length almost by accident, so the extension can be **fully additive**: no change to the base class’s existing behavior, and no change to any of the 30 already-registered message classes.

- `initStorage() / initCallbackStorage()` allocate exactly `_messageLength` bytes. Nothing requires `_messageLength` to mean “the fixed wire length” — it can mean “the maximum possible payload” instead.
- `ubxMessageVector::storePayload()` already does `if (len > msg->_messageLength) len = msg->_messageLength;` before the `memcpy`. It copies only the *actual* received length, clamped to the max. That is exactly correct once `_messageLength` is set to `UBX_NAV_SAT_MAX_LEN` (3068 bytes: 8-byte header + 255×12 -byte blocks).
- `extractFieldFrom(buffer, fieldName, value)` already takes an arbitrary buffer pointer — it does not have to be the start of the message.

So NAV-SAT can become an ordinary registered `ubxMessage` subclass, with a small, backward-compatible extension to describe “a header field table plus a repeated block field table,” rather than a parallel, special-cased mechanism.

2. Base-class extension (additive only)

2.1 New members on `ubxMessage`

All defaulted, so every existing message subclass is untouched:

```
const void *_blockFields = nullptr;    // nullptr => not a repeated-
block message
uint8_t _numBlockFields = 0;
uint16_t _blockHeaderLength = 0;       // bytes before the first
repeated block (8 for NAV-SAT)
uint16_t _blockLength = 0;             // bytes per block (12 for
NAV-SAT, 16 for NAV-SIG)
uint16_t _maxBlocks = 0;               // UBX_NAV_SAT_MAX_BLOCKS, for
the caller's loop bound
```

2.2 addClassID() gains 5 trailing, defaulted parameters

Every existing call site passes exactly today's 9 positional arguments and keeps compiling unchanged:

```
void addClassID(uint8_t Class, uint8_t ID, const char *classStr,
const char *idStr,
uint16_t messageLength, uint8_t numCallbackCopies,
uint8_t numFields,
const void *ubxFields, const uint32_t *msgOutKeys,
const void *blockFields = nullptr, uint8_t
numBlockFields = 0,
uint16_t blockHeaderLength = 0, uint16_t
blockLength = 0,
uint16_t maxBlocks = 0)
```

2.3 extractFieldFrom() gains two optional trailing parameters

```
bool extractFieldFrom(const uint8_t *buffer, const char *fieldName,
ubxAnyType *value,
const ubxField *fieldsOverride = nullptr,
uint8_t numFieldsOverride = 0) const
{
    const ubxField *fields = fieldsOverride ? fieldsOverride :
(const ubxField *)_fields;
    uint8_t numFields = fieldsOverride ? numFieldsOverride :
_numFields;
    // ...unchanged byte/bit-extraction switch below, now driven by
`fields`/`numFields`...
}
```

Every existing call (`extractFieldFrom(buffer, fieldName, value)`) is behaviorally identical. A new call for block data passes the block's own field table explicitly.

3. New generic accessors on DevUBLOXGNSS

Not NAV-SAT-specific, so NAV-SIG (and any future repeated-block message) gets this for free:

```
ubxAnyType getUbxMessageBlockField(ubxMessage *theMessage, uint16_t
blockIndex,
const char *fieldName);
ubxAnyType getUbxMessageBlockFieldCallback(ubxMessage *theMessage,
uint16_t blockIndex,
const char *fieldName);
```

Each computes the block's start address and delegates to `extractFieldFrom()`:

```
const uint8_t *blockBuffer = storage + theMessage->
_blockHeaderLength
+ (blockIndex * theMessage->
_blockLength);
theMessage->extractFieldFrom(blockBuffer, fieldName, &value,
(const ubxField *)theMessage->
_blockFields,
theMessage->_numBlockFields);
```

`getUbxMessageBlockField` reads from `_storage` (live/pollled read);
`getUbxMessageBlockFieldCallback` reads from `_callbackStorage` (frozen
callback read) — mirroring the existing `getUbxMessageField /`
`getUbxMessageFieldCallback` duality exactly.

Header-level fields (iTOW, version, numSvs) are unaffected and keep using the existing `getUbxMessageField / getUbxMessageFieldCallback`.

This flattens the sketch in AGENTS.md's example (`getNAVSATSVblock()` then `getUbxNavSatBlockData()`) into a single call per field. The per-block offset arithmetic is cheap enough that recomputing it on every field access isn't worth introducing an intermediate pointer/handle type.

4. New message class: `src/ubxMessages/ubxNAVSAT.h`

Follows the existing per-message subclass pattern used by all 30 registered messages:

- `messageLength = UBX_NAV_SAT_MAX_LEN` — the *maximum*, not a fixed wire size.
 - `ubxFields[]` — header-only table, 3 entries: iTOW, version, numSvs.
 - A new `ubxBlockFields[]` table, offsets relative to the start of one block: gnssId (0), svId (1), cno (2), elev (3), azim (4), prRes (6), flags (8, X4), plus the individual bitfields (qualityInd, svUsed, health, diffCorr, smoothed, orbitSource, ephAvail, almAvail, anoAvail, aopAvail, sbasCorrUsed, rtcmCorrUsed, slasCorrUsed, spartnCorrUsed, prCorrUsed, crCorrUsed, doCorrUsed) as startBit/bitWidth sub-fields of byte offset 8 — the same mechanism already used for NAV-STATUS's flags bitfields, just applied to a table addressed relative to `blockIndex` rather than the message start.
 - Constructor passes `_blockHeaderLength = 8, _blockLength = 12, _maxBlocks = UBX_NAV_SAT_MAX_BLOCKS`.
 - Ends with `ubxRegisterMessage(ubxNAVSAT);`.
-

5. Wiring into `processUBXpacket()` / `autoLookup()`

Once `ubxNAVSAT` self-registers, `ubxMessages.find(UBX_CLASS_NAV, UBX_NAV_SAT)` succeeds, so the existing registry branch in `processUBXpacket()` (the `if (ubxMessagePtr) { ubxMessages.storePayload(...); }` branch) handles NAV-SAT automatically. The old `else if (msg->id == UBX_NAV_SAT)` case (~35 lines, around line 2156 in `u-blox_GNSS.cpp` today) becomes dead code and should be deleted, along with the matching `UBX_NAV_SAT` case in `autoLookup()`.

6. Retiring the legacy v3 API

Consistent with how NAV-PVT was migrated (its old `packetUBXNAVPT` pointer and typed callback setter no longer exist anywhere in the codebase), this proposal retires the equivalent NAV-SAT machinery rather than keeping two parallel APIs:

- Remove `packetUBXNAVSAT` and its allocation/free code.
- Remove `UBX_NAV_SAT_t` (the wrapper struct with `automaticFlags`, `moduleQueried`, `callbackPointerPtr`, `callbackData`). Keep `UBX_NAV_SAT_data_t / _header_t / _block_t` — they remain a useful, documented description of the wire format even though nothing allocates them directly anymore.

- Remove `setAutoNAVSATcallbackPtr()` (the strongly-typed `void (*) (UBX_NAV_SAT_data_t *)` callback setter). Callbacks are registered through the existing generic `setAutoCallbackPtr()` mechanism instead, same as every other already-migrated message.
- Replace the body of `getNAVSAT()` with the same one-line thin wrapper used for every other registered message:

```
bool DevUBLOXGNSS::getNAVSAT(uint16_t maxWait)
{
    return getUBX(Class, ID, maxWait);
}
```

The public signature and return semantics (“true if new NAV-SAT data is available”) are unchanged, so existing sketches calling `getNAVSAT()` keep compiling.

7. Example callback, corrected

The callback sketch in AGENTS.md’s new section, rewritten against this API (also fixes the sketch’s own bug — `Serial.print()`; with no argument):

```
void newNAVSAT(ubxCallbackDataCommon_t *theData)
{
    ubxMessage *msg = myGNSS.getUbxMessagePtr(theData);

    uint8_t numSvs = myGNSS.getUbxMessageFieldCallback(msg, "numSvs");
    Serial.print(F("New NAV SAT data received. It contains data for
"));
    Serial.print(numSvs);
    Serial.println(numSvs == 1 ? F(" SV.") : F(" SVs.));

    for (uint8_t block = 0; block < numSvs; block++)
    {
        uint8_t gnssId = myGNSS.getUbxMessageBlockFieldCallback(msg,
block, "gnssId");
        switch (gnssId)
        {
            case 0: Serial.print(F("GPS    ")); break;
            case 1: Serial.print(F("SBAS   ")); break;
            case 2: Serial.print(F("Galileo ")); break;
            case 3: Serial.print(F("BeiDou  ")); break;
            case 4: Serial.print(F("IMES    ")); break;
            case 5: Serial.print(F("QZSS    ")); break;
            case 6: Serial.print(F("GLONASS ")); break;
            default: Serial.print(F("UNKNOWN ")); break;
        }

        uint8_t svId = myGNSS.getUbxMessageBlockFieldCallback(msg,
block, "svId");
        Serial.print(svId);
        if (svId < 10) Serial.print(F(" "));
        else if (svId < 100) Serial.print(F(" "));
        else Serial.print(F(" "));

        uint8_t cno = myGNSS.getUbxMessageBlockFieldCallback(msg, block,
"cno");
        for (uint8_t bar = 0; bar < cno; bar++)
            Serial.print(F("="));
        Serial.println();
    }
}
```

8. Scope

In scope: - NAV-SAT, as the worked example. - NAV-SIG as a direct follow-on: identical 8-byte-header + fixed-block shape, just `_blockLength = 16` and `_maxBlocks = UBX_NAV_SIG_MAX_BLOCKS` (92). No new mechanism needed — the same block-field-table approach applies unchanged.

Explicitly out of scope, per AGENTS.md's own deferral note ("We are not yet ready to add RXM-SFRBX and ESF-MEAS to the `ubxMessage` class. This will be added at a future date."): - RXM-SFRBX, ESF-MEAS. Both require `_numCallbackCopies > 1` (ring-buffered callback storage), which is a separate, orthogonal piece of design work — unrelated to the repeated-block mechanism proposed here, and not addressed by it.

Not addressed by this proposal: - Bounds-checking `blockIndex` against `_maxBlocks` / the header's actual `numSvs` inside `getUbxMessageBlockField[Callback]()` itself. The proposal leaves this to the caller, matching the loop pattern in AGENTS.md's own example (`for (block = 0; block < numSvs; block++)`), rather than adding a redundant runtime check on every field access. Flagged here as a judgment call that can be revisited. - RAM cost: `_callbackStorage` for NAV-SAT is allocated at the worst case (`UBX_NAV_SAT_MAX_LEN × _numCallbackCopies`), same as `_storage`. This is no worse than the v3 struct it replaces (which always allocated `blocks[UBX_NAV_SAT_MAX_BLOCKS]` regardless of how many SVs are actually visible), so it is not a regression, just carried forward.

9. Open questions

1. OK with fully retiring `packetUBXNAVSAT / setAutoNAVSATcallbackPtr` (a breaking change for any sketch using the old typed callback), matching how NAV-PVT was migrated?
2. OK with the flattened `getUbxMessageBlockField(msg, blockIndex, fieldName)` three-argument signature, instead of the two-step pointer-returning sketch in AGENTS.md?
3. Do NAV-SAT and NAV-SIG in the same pass, or NAV-SAT first with NAV-SIG as a separate, later follow-up?